

Guía FP Ejercicio

Tabla de multiplicar

Propósito de la guía. Acompañarte paso a paso para desarrollar el ejercicio de tabla de multiplicar, primero en PSeInt y luego en Code::Blocks, cuidando la lógica, la sintaxis y la prueba de escritorio. La idea es que no solo funcione, sino que entiendas por qué funciona.

Resultado esperado

Leer un número entero ingresado por teclado y mostrar su tabla de multiplicar del 1 al 10.

Contexto del ejercicio

En este ejercicio el programa solicita un número entero al usuario.

Luego, utilizando una estructura repetitiva Para, realiza multiplicaciones desde 1 hasta 10 y muestra cada resultado en pantalla.

La operación utilizada es:

$n \times i$

1. Seudocódigo para PSeInt

Antes de programar en C, conviene escribir la solución en PSeInt. Aquí se observa la lógica con palabras cercanas al lenguaje natural.

Proceso TablaMultiplicar

Definir n, i Como Entero

Escribir "Ingrese un número: "
Leer n

Para i <- 1 Hasta 10 Con Paso 1 Hacer

Escribir n, " x ", i, " = ", n * i

FinPara

FinProceso

Tip amigable: Imagina que: n guarda el número base de la tabla, i va cambiando desde 1 hasta 10.

En cada repetición el programa multiplica ambos valores y muestra el resultado.

2. Prueba de escritorio en PSeInt

La prueba de escritorio permite comprobar manualmente qué hace el algoritmo. Para que sea fácil de revisar, se muestra el comportamiento.

i	Operacion	Resultado
1	3 x 1	3
2	3 x 2	6
3	3 x 3	9
4	3 x 4	12
5	3 x 5	15
6	3 x 6	18
7	3 x 7	21
8	3 x 8	24
9	3 x 9	27
10	3 x 10	30

Salida esperada:

Ingrese un numero:

```

3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30

```

3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

PSeInt	C / Code::Blocks	Para recordar
Proceso ... FinProceso	int main() { ... return 0; }	Todo programa en C inicia en main.
Definir variables reales	int n, i;	Los números enteros se declaran con int.
Escribir	Printf();	Printf muestra mensajes y resultados.
Leer	Scanf();	Scanf permite ingresar datos.
Para i <- 1 Hasta 10	for(i = 1; i <= 10; i++)	El FOR tiene inicio, condición e incremento.

n * i	n * i	El operador de multiplicación en C es *.
-------	-------	--

Código validado en C para Code::Blocks

```
#include <stdio.h>

int main()
{
    int n, i;

    printf("Ingrese un numero: ");
    scanf("%d", &n);

    for(i = 1; i <= 10; i++)
    {
        printf("%d x %d = %d\n", n, i, n * i);
    }

    return 0;
}
```

Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.
5. Ingresar diferentes números enteros.
6. Comparar los resultados con la prueba de escritorio.

Errores comunes que debes evitar:

- Usar x en lugar de * para multiplicar en C.
- Olvidar el símbolo & en scanf.
- Declarar variables enteras como float.
- Colocar incorrectamente el incremento del for.
- Olvidar \n para separar líneas de la tabla.

4. Rúbrica de evaluación sobre 20 puntos

Esta rúbrica permite valorar el desarrollo completo del ejercicio, desde la comprensión del problema hasta la ejecución en Code::Blocks.

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Comprende correctamente el funcionamiento de la tabla de multiplicar.	Comprende parcialmente la lógica.	Confunde el uso del ciclo o la multiplicación.	4 pts
Seudocódigo en PSeInt	Usa correctamente variables y estructura para.	Presenta pequeños errores de sintaxis.	El algoritmo está incompleto.	4 pts

Fundamentos de Programación | Guía práctica

Prueba de escritorio	Comprueba correctamente los resultados.	Presenta una prueba parcial.	No evidencia seguimiento lógico.	4 pts
Traducción a C en Code::Blocks	El código compila y ejecuta correctamente.	Tiene errores menores corregibles.	El código no compila o falla.	5 pts
Presentación y orden	Trabajo limpio y organizado.	Comprensible con pequeños detalles.	Trabajo desordenado.	3 pts

Total: 20 puntos. Para una buena calificación, lo más importante es que exista coherencia entre pseudocódigo, prueba de escritorio, código en C y salida final.

Lista rápida de verificación antes de entregar

- El programa solicita un número entero.
- El ciclo FOR se ejecuta de 1 a 10.
- La multiplicación se realiza correctamente.
- Los resultados se muestran en orden.
- El código compila sin errores.
- La salida coincide con la prueba de escritorio.